

03. 의공파트 — 의료기기 수리 매뉴얼 챗봇 A RAG 챗봇

❖ 이 페이지 전제 — 복붙 전 1초 확인 (Sonnet도 그대로 동작하게 다 적었습니다) - 답변(채팅) 모델: `qwen3.6:35b-a3b` · 호출 `POST`
`http://121.138.151.6:11434/api/chat` - 검색(임베딩) 모델: `bge-m3:latest` · 호출 `POST` `http://121.138.151.6:11434/api/embeddings`
· body `{"model":"bge-m3:latest","prompt":"<문장>"}` → 응답의 `embedding` (단수 키) 이 1024차원 벡터. **조각이 8개면 이 단건 호출을 8번 루프로 돌립니다**(한 번에 하나씩 prompt에 넣어 호출). - 내 백엔드: FastAPI
`http://127.0.0.1:8000` — 프론트(HTML)는 반드시 이 백엔드만 부릅니다. - ❌ 금지: HTML/자바스크립트에서 `11434` · `ollama` 를 직접 호출하지 마세요(CORS·보안 사고).
모델 호출은 오직 백엔드(:8000)를 거칩니다. - 출발점: 공통 **STEP 0 기본코드** (`app.py` · `index.html`)가 떠 있는 상태에서 시작합니다(이 페이지에 코드를 다시 만들지 않음). - 🛠 **스트리밍 전제(중요)**: 공통 STEP 0 기본코드의 백엔드 `/api/chat` 응답은 **SSE 스트림**입니다 — 서버가 `data: {...}\n` 줄을 한 줄씩 흘려보냅니다(채팅 모델을 `stream:true` 로 부르기 때문). 프론트는 이 `data:` 라인을 한 줄씩 읽어 답을 이어 붙입니다. 아래 프롬프트 2에서 '출처'를 읽을 때도 이 스트림 위에 있습니다(스트림을 통째 JSON으로 바꾸지 않음).

오늘 할 일: 숙제에서 만든 “매뉴얼을 통째로 넣던” 챗봇을, **진짜 검색(임베딩·RAG)**으로 바꾸고 → **출처 표시 + 바이브↔하네스 토클** → **축** → **Playwright 자동 QA** 까지 있습니다. 숙제 못 했으면 아래 **STEP 0 캐치업**으로 출발선을 맞춥니다.

STEP 0 — 출발선 맞추기 (숙제 못 한 사람용 캐치업)

이 페이지의 모든 프롬프트는 **공통 STEP 0 기본코드**(`app.py` · `index.html`)가 떠 있는 상태에서 시작합니다. (이 페이지에 `app.py` / `index.html` 전체 코드를 다시 만들지 않습니다 — 공통 STEP 0 것을 그대로 씁니다.)

- **숙제를 한 사람**: 숙제에서 만든 매뉴얼 챗봇(프론트(HTML) → 백엔드 `http://127.0.0.1:8000` → 채팅 모델 `qwen3.6:35b-a3b`)이 그대로 출발점입니다. 그 위에 오늘 '검색(임베딩)'을 엽니다.
- **숙제를 못 한 사람**: 공통 STEP 0 기본코드(`app.py` · `index.html`)를 받아 띄우고 아래만 확인하면 출발선이 맞춰집니다.
 - 백엔드 터미널에 `http://127.0.0.1:8000` 서버가 떠 있다.
 - 브라우저로 `http://127.0.0.1:8000/health` 를 열면 정상 응답이 보인다.
 - 채팅 호출은 백엔드가 `POST http://121.138.151.6:11434/api/chat` (모델 `qwen3.6:35b-a3b`)로 한다. (HTML에서 `11434-ollama` 직접 호출 금지.)

🔧 **numpy 준비**: 오늘은 코사인 유사도 계산에 numpy를 씁니다. 터미널에서 `pip install numpy` 를 한 번 실행해 두세요. (이미 깔려 있으면 넘어가도 됩니다.)

✓ STEP 0 성공 확인: /health 가 정상 응답하고, 챗봇 화면에 아무 질문이나 넣어 답이 한 번 나오면 출발선 OK.

왜 검색이 필요한가: 매뉴얼이 8조각일 땐 통째로 넣어도 됩니다. 그런데 실제 매뉴얼은 수백 페이지 → 다 넣으면 느리고 답이 흐려져요. **질문과 가까운 조각만 골라** 넣는 게 RAG입니다. “E21 도어 오류”를 물으면 센서 코드 조각만 골라 넣는 식.

STEP 1에 넣을 내 자료 조각 (DOCS) — 8조각

각 조각을 한 덩어리(문자열)로 둡니다. 페이지 표기를 넣어두면 나중에 출처로 그대로 쓸 수 있습니다.

DOCS = [

"[폐색 알람 AL-OCC | p.42] 주입 중 폐색 경고음·정지. 점검: 라인 꺾임·눌림, 클램프 열림, 필터·삼방활전 막힘. 조치: 원인 해소 후 [재개], 지속 시 압력센서 점검·서비스 요청.",

"[압력 상승 AL-PRS | p.43] 주입압이 상한 초과 시 경고. 점검: 카테터 위치·환자 자세·라인 길이, 약액 점도. 조치: 원인 제거 후 상한 재설정, 반복 시 압력센서 캘리브레이션(자격 기술자).",

"[공기 알람 AL-AIR | p.45] 라인 내 기포 감지 시 정지. 점검: 챔버 액위, 프라임 상태, 센서 창 오염. 조치: 재프라이밍·기포 제거 후 재개, 센서 창 마른 천으로 닦기.",

"[배터리 점검·교체 | p.61] 완충 후 사용시간 표시 확인, 만충 대비 80% 미만이면 교체. 정품만 사용, 임의 분해 금지, 교체 후 충전 1사이클 권장.",

"[센서 오류 코드 | p.78] E12 공기감지센서 커넥터 접촉불량→재결합·재시작. E15 압력센서 불안정→캘리브레이션(자격자). E21 도어 닫힘 감지 실패→래치·자석 위치 확인.

E33 내부 과열→통풍구 확인·30분 냉각.",

"[세척·소독 | p.90] 외장: 전원 차단 후 중성세제 천으로 닦고 건조, 분무·침수 금지. 소독: 70% 알코올 천 가능, 화면·센서 창 강한 용제 금지. 환자 교체 시마다 외장 소독.",

"[정기 점검 | p.102] 일일: 알람음·화면·배터리 잔량. 월간: 압력센서 동작·도어 래치·충전. 연간: 압력센서 캘리브레이션·누설전류 측정(기록 보관).",

"[작업 안전 원칙 | p.7] 점검·조치는 자격 기술자 전용. 임의 분해·내부 수리·개조 금지. 환자 연결 상태 점검 금지(라인 분리 후). 의심 시 제조사 서비스 요청."

]

오늘의 테스트 질문: “폐색 알람 뜨면?” · “E21 무슨 뜻이야?” · “배터리 언제 갈아?” · “월간 점검 뭐 해?”

프롬프트 1 — DOCS 임베딩 + 가까운 조각만 검색 (STEP 1 핵심)

준비: numpy가 필요합니다 — 터미널에서 `pip install numpy` 를 먼저 한 번 실행하세요. 모든 모델 호출은 **백엔드(app.py, :8000)에서만** 합니다. index.html/자바스크립트에서 121.138.151.6:11434·ollama 직접 호출 금지.

지금 백엔드는 매뉴얼을 통째로 모델에 넘긴다. 이걸 '검색(RAG)'으로 바꿔줘.

– app.py 안에 위 매뉴얼 조각 리스트 DOCS 를 둔다.

- 각 조각을 임베딩으로 벡터화: POST

`http://121.138.151.6:11434/api/embeddings` , body `{"model":"bge-m3:latest","prompt":"<조각 문자열>"}`. 응답의 "embedding"(단수 키)이 1024 차원 벡터다. 조각이 8개이므로 이 단건 호출을 for 루프로 8번 돌려 8개 벡터를 만든다. (이 호출은 `app.py` 백엔드가 한다)

- 질문이 오면 질문도 같은 임베딩(`bge-m3:latest`, `/api/embeddings`, `prompt`에 질문 문자열, 응답 `embedding`)으로 벡터화하고, `numpy`로 코사인 유사도를 계산해 가장 가까운 조각을 점수 높은 순으로 정렬한다. 상위 2~3개(`top-k`)를 '항상' 컨텍스트로 넣는다. 임계값(`SIM_THRESHOLD=0.3`)은 '너무 무관한 잡음만 거르는' 낮은 값으로만 쓰고, 최상위 조각이 임계 미만이어도 최소 1개는 컨텍스트로 넘긴다(`top-k`가 비지 않게). 자료에 답이 실제로 있는지/없는지의 최종 판단은 임계값 숫자가 아니라 프롬프트 4의 시스템 프롬프트(모델)가 한다.

- 답변 생성(채팅) 호출: POST `http://121.138.151.6:11434/api/chat` , body `{"model":"qwen3.6:35b-a3b","messages":[...],"stream":true}`. 골라낸 조각만 컨텍스트로 넣어 한국어로 답하게 한다. (이 호출도 `app.py` 백엔드가 한다)

- 프론트는 :8000만 부른다. 응답은 SSE 스트림이다.)

- 첫 질문 때 한 번만 DOCS 8조각을 임베딩(단건 8회 루프)하고 메모리에 캐시한다 - 두 번째 질문부터는 다시 임베딩하지 않고 캐시된 8개 벡터를 재사용한다.

기대 결과 /  성공 확인: "E21 뜻이야?" 질문 시 답이 나오고, 백엔드 로그에 8조각 전체가 아니라 **2조각만** 컨텍스트로 들어간 게 보이면 성공(센서 코드 조각 p.78이 선택됨).

프롬프트 2 — 출처 표시 (페이지·항목)

 **전제(반드시 읽기):** 공통 STEP 0의 백엔드 `/api/chat` 응답은 SSE 스트림(`data: {...}\n` 줄을 흘림)입니다. 그래서 출처(sources)를 답변과 함께 보낼 때는 스트림을 통째 JSON으로 바꾸지 말고, **스트림 위에 한 줄을 더 엮습니다**. 구체적으로: 백엔드가 답 토큰을 다 흘린 뒤(또는 맨 앞에) `data: {"sources":[{"title":"센서 오류 코드","page":"p.78"}]}\n` 같은 **별도 sources 이벤트 라인**을 한 번 흘려보냅니다. 프론트는 `data:` 라인을 파싱하다가 `sources` 키가 있는 줄을 만나면 그걸로 출처 칩을 렌더링하고, 나머지(답 토큰) 줄은 지금처럼 화면에 이어 붙입니다.

답변에 어떤 조각을 근거로 썼는지 '출처'를 함께 줘. (현재 `/api/chat` 응답은 SSE 스트림 - `data:` 라인을 흘리는 방식이다. 이 전제를 깨지 말 것.)

- 고른 조각의 머리표([... | p.00])를 프론트에 함께 보낸다. 단, 스트림을 JSON 한 덩어리로 바꾸지 말고, 답 토큰을 다 흘린 뒤 별도 `sources` 이벤트 라인을 한 번 흘린다: `data: {"sources":[{"title":"센서 오류 코드","page":"p.78"}]}\n` (조각이 2개면 배열에 2개).

- `index.html`의 스트림 파서를 보강한다: `data:` 라인을 읽다가 `"sources"` 키가 있으면 그 배열로 출처 칩을 렌더링하고, 그 외 라인은 지금처럼 답 본문으로 이어 붙인다.

- `index.html`에서 답변 아래에 작게 "근거: 센서 오류 코드 (p.78)"처럼 출처 칩으로 표시.

-  칩에는 페이지 번호만 쓰지 말고 반드시 '조각 이름(머리표)'을 함께 표기.

(E12/E15/E21/E33는 모두 같은 'p.78' 한 조각이라, 페이지만 보이면 코드별로 다른 곳을 검색한 것처럼 오해된다. "근거: 센서 오류 코드 (p.78)"처럼 조각명을 같이 보여줄 것.)

- 고른 조각이 2개면 2개 다 표시.

- (호출 경로는 그대로: 프론트는 :8000만 부르고, 백엔드가 qwen3.6:35b-a3b를 POST http://121.138.151.6:11434/api/chat 로 호출. HTML 직접 호출 금지.)

기대 결과 /  성공 확인: 답 아래에 “근거: 센서 오류 코드 (p.78)”처럼 **조각 이름 + 페이지**가 함께 적힌 칩이 뜨면 성공.

프롬프트 3 — 바이브 ↔ 하네스 토글 (오늘의 하이라이트)

화면에 '바이브 ↔ 하네스' 토글 버튼을 추가해줘.

- 바이브(off): 검색·매뉴얼 없이 모델만 호출(출처 없음, 일반론).
- 하네스(on): 검색(RAG) + 출처 표시 + 안전 고지.
- 요청 보낼 때 {bare: true/false}로 구분.
- 두 모드 다 호출 경로는 동일: 프론트(HTML)는 백엔드(:8000)만 부르고, 백엔드가 qwen3.6:35b-a3b를 POST http://121.138.151.6:11434/api/chat 로 호출한다(바이브여도 프론트가 11434를 직접 부르지 않음).
- 화면 위에 지금 모드가 뭔지 배지로 보이게 ("바이브 모드"/"하네스 모드").

기대 결과 /  성공 확인: “E21 뜻이야?”를 두 모드로 물으면 — 바이브는 출처·고지 없이 두루뭉술, 하네스는 매뉴얼 근거+출처(센서 오류 코드 p.78)+안전 고지. 배지로 모드가 바뀌는 게 보이면 성공.

프롬프트 4 — 안전 고지 자동 + 자료 밖 차단 (하네스 규칙)

하네스 모드의 답변 규칙을 강화해줘.

1) 점검·조치 답변 끝에 항상 이 고지를 붙인다(바이브 모드엔 붙이지 않음):

"⚠️ 자격 기술자 전용 — 임의 분해·내부 수리 금지, 환자 연결 상태 점검 금지, 의ishi 제조사 서비스 요청."

2) '자료 밖 차단'은 임계값 숫자로 하지 말고 시스템 프롬프트(모델)가 판단하게 한다. 시스템 프롬프트에 이렇게 넣는다: "아래 근거 자료에 질문의 답이 실제로 있으면 출처와 함께 답하고, 자료에 답이 없으면 추측하지 말고 '자료에서 확인되지 않습니다'라고만 답하라." → 이러면 '물 끓이는 설비 있는 곳?'처럼 자료에 답이 있는 질문은 정상 답변되고, '안식년'처럼 의미는 가까워도(코사인 0.49 등) 자료에 규정이 없는 질문은 모델이 '확인되지 않습니다'로 막는다.

- 임계값(SIM_THRESHOLD = 0.3)은 app.py 상단에 두되(파일 맨 위, DOCS 근처) '너무 무관한 잡음만 거르는' 1차 필터로만 쓴다. 최상위 조각이 임계 미만이어도 top-k가 비지 않도록 최소 1개는 컨텍스트로 넘긴다.

- ⚠️ 임계값을 0.4~0.6으로 올려 '규정 밖 질문'을 차단하려 하지 말 것. 실측상 규정에 없는 질문도 의미가 가까우면 코사인이 0.45~0.50까지 나와 임계값으로는 걸려지지 않고(차단 실패), 반대로 임계값을 올리면 정상 질문까지 놓친다(정상 질문 누락). 차단은 임계값 단독이 아니라 시스템 프롬프트가 맡는다.

3) 사용자가 분해·개조·직접 수리 방법을 물으면 절차를 알려주지 말고 "자격 기술자/제조사 서비스 대상"으로 안내.

- (규칙 강화만 하는 작업. 호출은 그대로 백엔드(:8000) → qwen3.6:35b-a3b POST http://121.138.151.6:11434/api/chat. HTML 직접 호출 금지.)

기대 결과 /  성공 확인: 매뉴얼에 답이 있는 질문(예: “E21 무슨 뜻이야?”)은 출처와 함께 답이 뜨고, 매뉴얼에 규정이 없는 질문(예: 안식년·휴가 같은 자료 밖 질문)은 의미가 가까워 검색은 되더라도 모델이 “자료에서 확인되지 않습니다”로 막으면 성공. 하네스 답 끝엔 “⚠️ 자

격 기술자 전용 ...” 고지가 자동으로 붙고, “직접 분해법”은 거절된다. 바이브 답변 고지가 없어 비교가 된다.

프롬프트 5 — 오류 코드 빠른 검색 패널

오류 코드(E12/E15/E21/E33)를 자주 찾으니 전용 패널을 만들어줘.

- 화면 위에 코드 입력칸 + [코드 검색] 버튼, 그리고 코드 칩(E12/E15/E21/E33).
 - 칩을 누르거나 코드를 입력하면 "오류 코드 00의 의미와 조치를 매뉴얼 근거로 알려줘"
- 질문을 하네스 모드로 자동 전송(프론트는 백엔드 :8000으로만 보낸다 - 11434·ollama 직접 호출 금지).
- 결과에 의미·조치·출처(조각명 + p.78)·안전 고지가 다 포함되게.
 - 매뉴얼에 없는 코드면(자료에 그 코드 규정이 없으면) 시스템 프롬프트 판단으로 "미수록 코드 - 자료에서 확인되지 않습니다"로 답하게(임계값으로 막지 말 것).

기대 결과 /  성공 확인: “E33” 칩을 누르면 화면에 “내부 과열 → 통풍구 확인·30분 냉각” + 출처 칩 “근거: 센서 오류 코드 (p.78)” + 안전 고지가 모두 보이면 성공. 없는 코드는 “미수록”.

프롬프트 6 — 흑으로 실수 자동 차단 (STEP 3) 에이전트

.claude/settings.json 에 넣을 흑을 JSON으로 만들어줘:

- 1) PreToolUse: HTML 파일에 "11434" 또는 "ollama" 직접 호출이 들어가면 차단 (프론트→백엔드 분리 위반 방지).
 - 2) PostToolUse: app.py가 수정되면 "백엔드 점검: /health 먼저 확인" 메시지 출력.
 - 3) Stop: 작업 끝나면 git status 보여주기.
- 복붙 가능한 형식으로.

기대 결과: index.html에 ollama 주소를 실수로 넣으면 흑이 막고 이유를 알려준다.

프롬프트 7 — Playwright 자동 QA (STEP 4) 에이전트

Playwright로 내 매뉴얼 챗봇을 자동 테스트해줘.

- 호출 경로 전제: 프론트(HTML)는 백엔드(http://127.0.0.1:8000)만 부르고, 백엔드가 qwen3.6:35b-a3b를 POST http://121.138.151.6:11434/api/chat 로 호출한다(HTML에서 11434·ollama 직접 호출 금지).
- 1) 백엔드 http://127.0.0.1:8000/health 연결 확인.
 - 2) 브라우저로 index.html 열기.
 - 3) 하네스 모드인지 확인(아니면 토글 켜기).
 - 4) 질문칸에 "E21 무슨 뜻이야?" 입력 후 전송.
 - 5) 답이 뜨고(35b 모델·스트리밍 첫 토큰 지연 감안: 타임아웃 15~20초로 넉넉히), 답에 "도어"가 포함 + 출처에 "센서 오류 코드" 및 "p.78" 표시 + 안전 고지("자격 기술자") 포함되는지 확인.
 - 6) 셋 다 통과면 "통과", 하나라도 빠지면 "실패"로 리포트.
- 테스트 코드(tests/qa.spec.ts)와 실행 방법도 같이.

기대 결과 /  성공 확인: 답·출처(센서 오류 코드 p.78)·안전 고지 3종이 다 있으면 터미널에 “통과” 리포트가 출력되면 성공.

프롬프트 8 — 바이브↔하네스 비교를 자동으로 (검증·마무리)

같은 질문을 두 모드로 자동 비교하는 테스트를 추가해줘.

– 두 모드 모두 요청은 백엔드(<http://127.0.0.1:8000>) 경유(HTML에서 11434·ollama 직접 호출 금지). 백엔드는 qwen3.6:35b-a3b를 POST <http://121.138.151.6:11434/api/chat> 로 호출.

- 1) 질문 "E21 무슨 뜻이야?"를 바이브 모드(bare:true)로 보내 답 A를 받는다.
- 2) 같은 질문을 하네스 모드(bare:false)로 보내 답 B를 받는다.
- 3) 하네스 답(B)에만 출처("센서 오류 코드" p.78)와 안전 고지가 있고, 바이브 답(A)엔 없는지 확인. (두 응답 모두 SSE 스트림이므로, 출처는 프롬프트 2의 별도 sources 이벤트 라인에서 읽는다 — 하네스만 그 라인이 흐른다.)
- 4) 결과를 "하네스=근거+출처 / 바이브=출처 없음"으로 한 줄 리포트.

기대 결과 /  성공 확인: 터미널 리포트에 하네스에만 출처·고지가 붙은 게 확인되면(“하네스=근거+출처 / 바이브=출처 없음”) 성공 → “오늘의 차이”가 증명된다.

마무리 · 안전

- **점검:** 바이브↔하네스 차이(근거·출처·안전 고지)가 보이면 오늘 성공.
- 이 챗봇 답은 **참고용** — 실제 점검·수리는 **자격 기술자**, 의심 시 **제조사 서비스**.
- **임의 분해·내부 수리·개조 금지**, 환자 연결 상태 점검 금지.
-  **분리 강제:** HTML/자바스크립트에서 모델 서버 (121.138.151.6:11434 · ollama)를 **직접 호출하지 마세요**. 모델 호출은 반드시 내 백엔드(<http://127.0.0.1:8000>)를 경유합니다(CORS·보안 사고 방지).
- 자료는 **샘플 발체만** — 실제 장비 일련번호·환자정보·내부 보안자료 입력 금지. 모델 서버 주소(121.138.151.6:11434)는 외부 공유 금지.
- 다음: 실제 매뉴얼을 같은 형식 조각으로 늘려 넣기(파일 업로드는 추후), 리콜 대조(B)와 합치기 — 이때도 **건수·대상 로트 같은 정확 대조는 백엔드 파이썬 코드가 결정론적 정답값을 계산해** 컨텍스트로 넘기고, **챗봇은 그 값을 인용해 자연어로만** 답한다(모델이 숫자를 세지 않음 — 환각 방지). 검색·계산은 코드, 답변 표현은 챗봇.